

Animal Shelter Simulator – Project 1 Design Document

Author: Owen Lindsey

Instructor: Professor David Parker

Course: CST-180: Python Programming 1

Institution: Grand Canyon University

Date: October 26, 2025

Summary

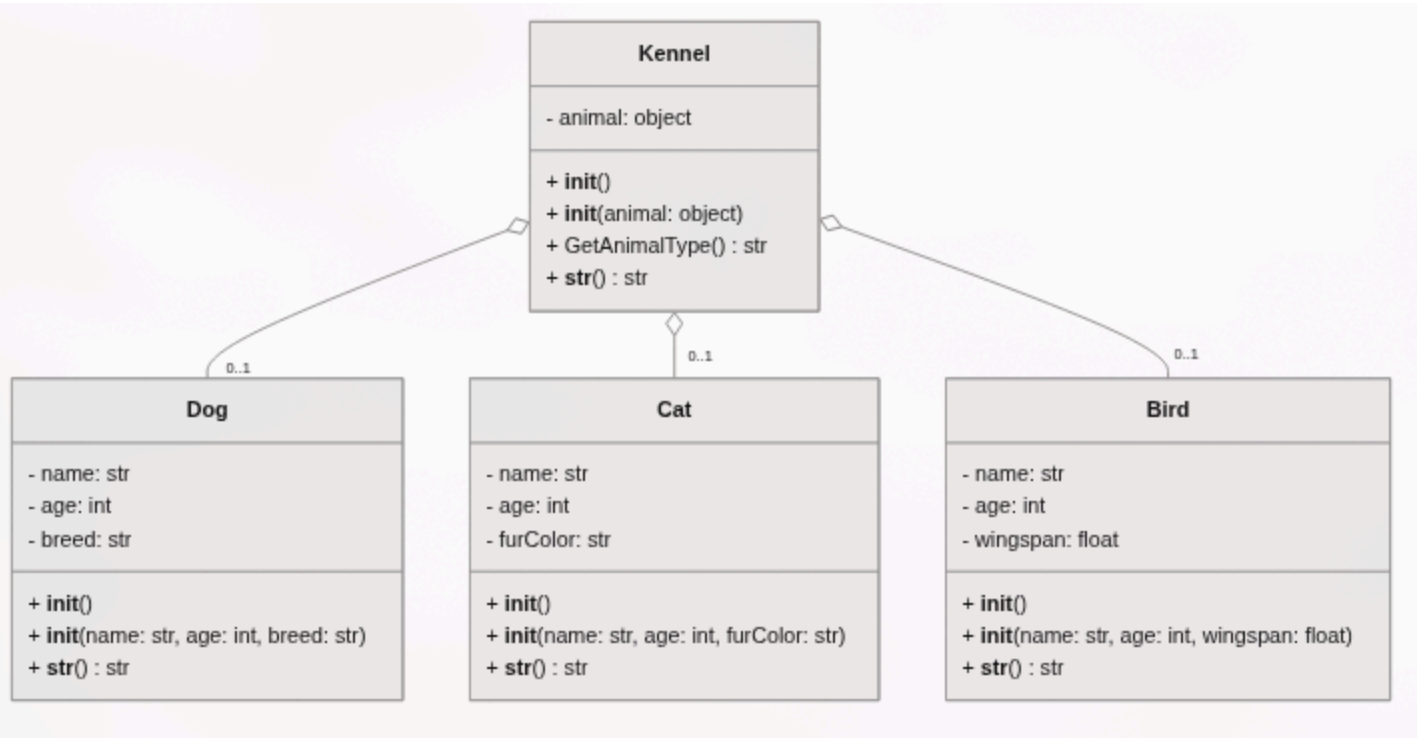
This first project establishes the object model and basic design for an animal shelter simulator without inheritance. It defines three independent animal classes (`Dog` , `Cat` , `Bird`) with required attributes and behaviors, plus a `KenneL` class that contains exactly one animal (a `Dog` , `Cat` , or `Bird`) at a time. The `KenneL` can report the contained animal's type using the object's `__class__.__name__` (i.e., the class name string). The deliverables include a UML class diagram, an IPO summary, detailed pseudocode and a flowchart for the program logic, sample data, and corresponding sample output.

Input / Processing / Output

- **Inputs:**
 - User or test-provided animal details:
 - `Dog` : Name (str), Age (int), Breed (str)
 - `Cat` : Name (str), Age (int), Fur Color (str)
 - `Bird` : Name (str), Age (int), Wingspan (float or str units)
 - Choice of which animal instance to place into a `KenneL` .
 - **Processing:**
 - Construct animal objects using an overloaded constructor (supporting both no-arg defaults and full-arg initialization).
 - Generate human-readable object text using an overloaded `__str__` method.
 - Place exactly one animal into a `KenneL` (containment).
 - Determine the contained animal type via `obj.__class__.__name__` .
 - **Outputs:**
 - Printed or displayed string representations of animals.
 - The `KenneL` string, showing which animal it contains.
 - The reported animal type from `KenneL.GetAnimalType()` .
-

UML Class Diagram (no inheritance; containment only)

Note: The diagram purposely avoids inheritance. `Kennel` has a containment relationship to one of `Dog` , `Cat` , or `Bird` .



Textual notes:

- No inheritance hierarchy is used; each animal is a standalone class.
 - `Kennel` composes/contains exactly one animal at a time (can be empty).
 - `GetAnimalType` returns the class name string of the contained object.
-

Design – Detailed Pseudocode

High-level program flow to construct animals, place them in a kennel, and report details.

```
BEGIN
  DEFINE class Dog with attributes (name, age, breed)
    PROVIDE overloaded constructors:
      - __init__() sets default values
      - __init__(name, age, breed) sets provided values
    PROVIDE __str__() that returns formatted details

  DEFINE class Cat with attributes (name, age, furColor)
    PROVIDE overloaded constructors (no-arg, full-arg)
    PROVIDE __str__()

  DEFINE class Bird with attributes (name, age, wingspan)
    PROVIDE overloaded constructors (no-arg, full-arg)
    PROVIDE __str__()

  DEFINE class Kennel with attribute (animal)
    PROVIDE overloaded constructors:
      - __init__() initializes animal to None
      - __init__(animal) sets provided Dog/Cat/Bird
    METHOD GetAnimalType():
      IF animal is None THEN return "None"
      ELSE return animal.__class__.__name__
    METHOD __str__():
      IF animal is None THEN return "Kennel(empty)"
      ELSE return "Kennel(" + animal.__str__() + ")"

  // Sample execution logic
  CREATE dog1 as Dog("Buddy", 3, "Labrador")
  CREATE cat1 as Cat("Mittens", 2, "Tabby")
  CREATE bird1 as Bird("Sky", 1, 0.35)

  CREATE kennel as Kennel()

  // Place an animal into the kennel (e.g., dog1)
  SET kennel.animal = dog1
  PRINT kennel.__str__()
  PRINT kennel.GetAnimalType()

  // Replace with another animal (e.g., cat1)
  SET kennel.animal = cat1
  PRINT kennel.__str__()
  PRINT kennel.GetAnimalType()

  // Replace with a bird
  SET kennel.animal = bird1
  PRINT kennel.__str__()
  PRINT kennel.GetAnimalType()
END
```

Python snippet showing expected method shapes:

```
class Dog:
    def __init__(self, name: str = "", age: int = 0, breed: str = ""):
        self.name = name
        self.age = age
        self.breed = breed
    def __str__(self) -> str:
        return f"Dog(Name={self.name}, Age={self.age}, Breed={self.breed})"

class Cat:
    def __init__(self, name: str = "", age: int = 0, furColor: str = ""):
        self.name = name
        self.age = age
        self.furColor = furColor
    def __str__(self) -> str:
        return f"Cat(Name={self.name}, Age={self.age}, FurColor={self.furColor})"

class Bird:
    def __init__(self, name: str = "", age: int = 0, wingspan: float = 0.0):
        self.name = name
        self.age = age
        self.wingspan = wingspan
    def __str__(self) -> str:
        return f"Bird(Name={self.name}, Age={self.age}, Wingspan={self.wingspan})"

class Kennel:
    def __init__(self, animal: object | None = None):
        self.animal = animal
    def GetAnimalType(self) -> str:
        return "None" if self.animal is None else self.animal.__class__.__name__
    def __str__(self) -> str:
        return "Kennel(empty)" if self.animal is None else f"Kennel({self.animal})"
```

Sample Data

Dog: Name = "Buddy", Age = 3, Breed = "Labrador"

Cat: Name = "Mittens", Age = 2, Fur Color = "Tabby"

Bird: Name = "Sky", Age = 1, Wingspan = 0.35

Sample Output (using the sample data)

```
Kennel(Dog(Name=Buddy, Age=3, Breed=Labrador))
Dog
Kennel(Cat(Name=Mittens, Age=2, FurColor=Tabby))
Cat
Kennel(Bird(Name=Sky, Age=1, Wingspan=0.35))
Bird
```

